K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

And

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Data Structures and Algorithms -3 (25CS2103E)

RÉSUMÉ–JOB MATCHING & TALENT MARKETPLACE ENGINE

Submitted by

J. Sohan – 2520080060

J. Sricharan – 25200900137

Team Number: 26

Section: 12

Under the guidance of

CHANDHUSHA KANDA

CS&IT

Hyderabad – 500090, Telangana, India.

DECLARATION

We hereby declare that the project report entitled “Résumé–Job Matching & Talent Marketplace Engine” is a bonafide work carried out by J. Sohan (2520080060) and J. Sricharan (25200900137) in partial fulfilment of the requirements for the course Data Structures and Algorithms – 3 (25CS2103E) at K L (Deemed to be) University during the academic year 2026–2027. The work presented in this report is based on the project methodology, algorithms and system design developed for this project and has not been copied from another department, university or institute.

J. Sohan
2520080060

J. Sricharan
25200900137

CERTIFICATE

This is to certify that the project report entitled “Résumé–Job Matching & Talent Marketplace Engine” is a bonafide work done and submitted by J. Sohan (2520080060) and J. Sricharan (25200900137) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence & Data Science at K L (Deemed to be) University during the academic year 2026–2027.

Signature of the Supervisor	Signature of the HOD	Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported us in completing this project. We are especially thankful to our project guide, Chandusha Kanda, CS&IT, for the guidance, encouragement and valuable suggestions provided throughout the project.

We also express our gratitude to the Department of Artificial Intelligence & Data Science, K L (Deemed to be) University, for providing the facilities and academic environment required to carry out this Data Structures and Algorithms – 3 project.

We thank all teaching and non-teaching staff members, classmates and friends who assisted us directly or indirectly. Finally, we are grateful to our parents for their constant support and encouragement.

J. Sohan – 2520080060

J. Sricharan – 25200900137

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1. ABSTRACT

The Résumé–Job Matching & Talent Marketplace Engine is an algorithm-driven recruitment platform designed to improve the efficiency of candidate screening, job recommendation and project team formation. The system processes résumé and job-posting text, extracts relevant skills, normalizes skill names and compares candidate profiles with job requirements to calculate compatibility. Exact skill and phrase search is supported through KMP and the Z-Function, while Rabin–Karp provides hash-based keyword searching. Levenshtein/Damerau-Levenshtein Dynamic Programming handles spelling and formatting variations in skill names. Suffix Arrays with LCP support indexed text similarity and repeated prefix/suffix queries. For recruitment optimization, candidates and jobs are represented as a bipartite graph and matching/max-flow techniques are used to select compatible candidate-to-role assignments. At the project level, a Greedy Set Cover approximation selects a small set of candidates whose combined skills cover required project capabilities. The overall pipeline therefore connects string algorithms, dynamic programming, hashing, graph algorithms and approximation algorithms into a single talent marketplace workflow.

Team Name: Team 26

Team Members: J. Sohan (2520080060); J. Sricharan (25200900137)

Course: Data Structures and Algorithms – 3 (25CS2103E)

2. INTRODUCTION

2.1 Background

Modern recruitment systems handle large volumes of résumés and job opportunities. Manual screening becomes time-consuming when many candidates use different names, spellings or representations for similar skills. An algorithm-driven marketplace can automate the repetitive parts of screening and provide a consistent basis for candidate ranking and job recommendation.

2.2 Motivation

- Reduce the effort required to screen large résumé and job collections.
- Normalize and compare skill names even when their textual representations differ.
- Provide compatibility scores that can support candidate ranking and job recommendation.
- Use graph-based optimization to assign suitable candidates to available roles.
- Extend individual matching to project-level team formation using skill coverage.

2.3 Proposed System

The proposed engine maps each recruitment task to a suitable advanced algorithm. Résumé/job text is treated as the input. KMP and the Z-Function perform exact phrase matching, Rabin–Karp supports hash-based searching, Edit Distance DP performs fuzzy skill-name matching, Suffix Array + LCP support indexed similarity queries, Bipartite Matching + Max-Flow perform candidate-to-role assignment, and Greedy Set Cover provides an approximate minimum team for project skill coverage.

2.4 Scope

The project focuses on the algorithmic core of a talent marketplace: text indexing, skill matching, ranking, role assignment and team formation. The PPT specifies a web-oriented technology stack consisting of a frontend, backend and database, but the supplied project data does not define a final deployed application architecture or measured production dataset.

3. LITERATURE SURVEY

3.1 Existing Recruitment Platforms and Approaches

LinkedIn Jobs

LinkedIn provides job search and job-match functionality in which qualifications from a profile/resume are compared with job requirements. This demonstrates the practical value of skill-based matching and ranking.

Reference: https://www.linkedin.com/jobs/

Indeed

Indeed supports job-search preferences based on qualifications such as work experience, skills, education and certifications, and presents match information in job-search results. This is conceptually related to the proposed candidate–job compatibility workflow.

Reference: https://www.indeed.com/

Workable

Workable provides applicant tracking, searchable candidate information, candidate evaluation and AI-powered screening features. It illustrates how automated candidate evaluation can be integrated into a recruiting workflow.

Reference: https://www.workable.com/

Traditional keyword screening

Basic keyword screening searches résumés for required terms. The proposed project extends this idea by combining exact string matching, hashing and fuzzy matching so that skill variations can be handled more systematically.

Reference: Algorithmic approach used as the project baseline.

Graph-based assignment

Candidate-to-role assignment can be represented as a bipartite graph. Matching and flow algorithms can then select compatible assignments while considering the availability of multiple roles.

Reference: Algorithmic approach used in the proposed system.

3.2 Gap Identified

The project combines several algorithmic techniques in one recruitment pipeline rather than relying only on keyword search. The distinctive focus is the integration of string

algorithms, dynamic programming, hashing, suffix indexing, graph matching and approximation algorithms for both individual hiring and project-level team formation.

4. PROBLEM STATEMENT AND OBJECTIVES

4.1 Problem Statement

Design an algorithm-driven talent marketplace that efficiently matches résumés to job requirements, ranks compatible candidates, assigns candidates to roles, and forms compact project teams based on required skill coverage.

4.2 Problem Description

Large candidate and job collections make manual screening inefficient, while inconsistent skill names make simple keyword matching unreliable. The system therefore needs efficient text matching, fuzzy similarity, ranking and combinatorial optimization techniques.

4.3 Objectives

- Extract and search skill phrases efficiently from résumé and job text.
- Normalize skill names and handle spelling or formatting variations.
- Calculate compatibility and rank candidates for jobs.
- Recommend suitable jobs to candidates based on skill alignment.
- Model candidates and roles as a bipartite graph for optimized assignment.
- Select a small candidate team whose combined skills cover project requirements.
- Demonstrate the practical application of advanced DSA concepts to a real-world recruitment problem.

4.4 Key Inputs and Outputs

Input	Processing	Output
Résumé text	Skill/phrase matching and normalization	Extracted candidate skills
Job description	Requirement matching	Required skill set
Candidate + job skills	Similarity and scoring	Compatibility score / ranking
Candidates + roles	Bipartite matching / max-flow	Role assignments
Project requirements + candidates	Greedy set cover approximation	Compact skill-covering team

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

- Processor: Intel Core i3 or above.
- RAM: Minimum 4 GB; 8 GB recommended.
- Storage: Minimum 10 GB free space.
- Internet connection for online services and deployment.
- Standard display, keyboard and mouse.

5.2 Software Requirements

- Programming: Java / Python.
- Frontend: HTML, CSS, JavaScript / React.
- Backend: Spring Boot / Flask.
- Database: MySQL / MongoDB.
- IDE: VS Code / IntelliJ / PyCharm.
- Version control: Git / GitHub.
- Browser: Chrome / Edge.

5.3 Recommended Project Environment

For a student implementation, Java or Python can be used for the algorithmic modules. A React or HTML/CSS/JavaScript frontend can collect résumé and job inputs, while Spring Boot or Flask can expose the matching functions as APIs. MySQL or MongoDB can store candidate, job and skill information. Git/GitHub can be used for version control.

6. IMPLEMENTATION

6.1 Overall Algorithm Pipeline

1. Résumé / Job Text → Input documents
2. KMP + Z-Function → Fast exact skill / phrase matching
3. Rabin–Karp → Hash-based candidate/job keyword search
4. Edit Distance DP → Fuzzy skill-name similarity
5. Suffix Array + LCP → Efficient repeated similarity / prefix queries
6. Bipartite Matching + Max-Flow → Optimal candidate-to-role assignment
7. Greedy Set Cover Approximation → Minimum / near-minimum team skill coverage

6.2 Module 1 – KMP Exact Skill Matching

KMP is used to locate exact skill or phrase occurrences in résumé/job text. A prefix-function/LPS table is built for the pattern so that mismatches do not require restarting the search from the beginning.

```
// Representative Java implementation aligned with the project
methodology
static int kmpSearch(String text, String pattern) {
    int[] lps = new int[pattern.length()];
    for (int i = 1, len = 0; i < pattern.length();) {
        if (pattern.charAt(i) == pattern.charAt(len))
            lps[i++] = ++len;
        else if (len > 0)
            len = lps[len - 1];
        else
            lps[i++] = 0;
    }

    for (int i = 0, j = 0; i < text.length();) {
        if (text.charAt(i) == pattern.charAt(j)) {
            i++; j++;
            if (j == pattern.length()) return i - j;
        } else if (j > 0) j = lps[j - 1];
        else i++;
    }
    return -1;
}
```

6.3 Module 2 – Rabin–Karp Keyword Search

Rabin–Karp computes rolling hashes for text windows and compares a pattern hash with each window. Double hashing can be used to reduce the risk of collisions when many skill phrases are searched.

```
// Representative rolling-hash search
static boolean rabinKarp(String text, String pat) {
    int m = pat.length(), n = text.length();
    if (m > n) return false;

    long base = 257, mod = 1_000_000_007L;
    long ph = 0, wh = 0, power = 1;

    for (int i = 0; i < m; i++) {
        ph = (ph * base + pat.charAt(i)) % mod;
        wh = (wh * base + text.charAt(i)) % mod;
        if (i < m - 1) power = (power * base) % mod;
    }

    for (int i = 0; i <= n - m; i++) {
        if (ph == wh && text.regionMatches(i, pat, 0, m)) return
true;
        if (i < n - m) {
            wh = (wh - text.charAt(i) * power % mod + mod) % mod;
            wh = (wh * base + text.charAt(i + m)) % mod;
        }
    }
    return false;
}
```

6.4 Module 3 – Edit Distance Dynamic Programming

Levenshtein distance measures the minimum number of insertions, deletions and substitutions needed to transform one skill name into another. Damerau-Levenshtein can additionally consider adjacent transpositions. A small distance can be interpreted as evidence that two skill labels are variants of one another.

```
static int editDistance(String a, String b) {
    int n = a.length(), m = b.length();
    int[][] dp = new int[n + 1][m + 1];

    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int cost = a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1;
            dp[i][j] = Math.min(
                Math.min(dp[i - 1][j] + 1, dp[i][j - 1] + 1),
                dp[i - 1][j - 1] + cost
            );
        }
    }
    return dp[n][m];
}
```

6.5 Module 4 – Bipartite Matching and Greedy Set Cover

Candidates and jobs are represented as two partitions of a bipartite graph. An edge represents a compatibility relationship. Matching or max-flow can then select assignments. For team formation, each candidate covers a subset of required skills; a greedy set-cover procedure repeatedly chooses the candidate that covers the largest number of currently uncovered skills.

```
// Simplified greedy set-cover idea
Set<String> uncovered = new HashSet<>(requiredSkills);
List<Candidate> team = new ArrayList<>();

while (!uncovered.isEmpty()) {
    Candidate best = null;
    int bestGain = 0;

    for (Candidate c : candidates) {
        int gain = 0;
        for (String s : c.skills)
            if (uncovered.contains(s)) gain++;
        if (gain > bestGain) {
            bestGain = gain;
            best = c;
        }
    }

    if (best == null) break; // remaining skills cannot be covered
    team.add(best);
    uncovered.removeAll(best.skills);
}
```

6.6 Data Flow / System Architecture

A practical deployment can be organized as: User → Frontend → Backend/API → Algorithm Engine → Database. The frontend collects résumé and job data. The backend validates requests and invokes the algorithm modules. The algorithm engine performs extraction, normalization, similarity scoring and optimization. The database stores candidate profiles, job postings, skills and matching results.

6.7 Algorithm-to-Task Mapping

Algorithm	Task	Purpose	Expected Benefit
KMP / Z-Function	Exact pattern search	Find exact skills/phrases	Efficient exact matching
Rabin–Karp	Hash-based search	Search multiple skill phrases	Fast indexed-style keyword search
Edit Distance DP	Fuzzy matching	Compare skill-name variants	Handles spelling/format differences

Suffix Array + LCP	Text similarity	Repeated prefix/suffix queries	Efficient indexed text comparison
Bipartite Matching / Max-Flow	Role assignment	Assign candidates to jobs	Optimized compatible assignments
Greedy Set Cover	Team formation	Cover project skills	Small/near-minimum team

7. RESULTS & DISCUSSION

7.1 Functional Results

The designed pipeline provides a complete algorithmic flow from résumé/job text to matching and team formation. Exact matching identifies explicitly stated skills, fuzzy matching handles naming variations, and graph/approximation methods extend the system from individual candidate ranking to role assignment and team selection.

Test Area	Expected Output	Discussion
Exact skill search	Skill found / not found	KMP and Z-Function support direct phrase matching.
Keyword search	Matching résumé/job records	Rabin–Karp provides rolling-hash search.
Skill normalization	Similarity / distance value	Edit Distance supports fuzzy comparison.
Candidate ranking	Ordered candidate list	Compatibility scores provide a ranking basis.
Role assignment	Candidate-to-role pairs	Bipartite matching / max-flow handles assignment.
Team formation	Selected candidate set	Greedy set cover targets project skill coverage.

7.2 Discussion

The main benefit of the proposed design is the division of recruitment work into algorithmically appropriate subproblems. String algorithms are used where text search is needed, dynamic programming is used for fuzzy similarity, graph algorithms are used for assignment, and approximation is used for the NP-hard-style team coverage problem. This modular structure makes the system suitable for demonstrating advanced DSA concepts within a realistic application domain.

7.3 Screenshots to Insert

- Screenshot 1 – Résumé input / candidate profile screen.
- Screenshot 2 – Job posting / job requirement screen.
- Screenshot 3 – Candidate ranking with compatibility scores.
- Screenshot 4 – Candidate-to-role assignment result.

- Screenshot 5 – Project team recommendation and covered skills.

Note: The supplied PPT does not contain actual application screenshots or measured numerical performance results. Therefore, the above items are placeholders for the project's real execution screenshots and should be replaced with the team's captured outputs before final submission.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Résumé–Job Matching & Talent Marketplace Engine demonstrates how advanced Data Structures and Algorithms can be applied to recruitment. KMP and the Z-Function support exact pattern matching; Rabin–Karp enables hash-based search; Levenshtein/Damerau-Levenshtein Dynamic Programming handles fuzzy skill-name matching; Suffix Arrays and LCP support indexed text-similarity operations; Bipartite Matching and Max-Flow optimize candidate-to-role assignment; and Greedy Set Cover approximation helps construct compact teams with broad project skill coverage. The project therefore connects multiple DSA concepts into a single end-to-end recruitment workflow.

8.2 Future Scope

- Add stronger résumé parsing and automatic skill extraction from structured and unstructured documents.
- Introduce richer skill normalization using a maintained skill taxonomy or ontology.
- Add explainable matching so users can see which skills increased or reduced a compatibility score.
- Improve ranking with additional factors such as experience, education and role preferences.
- Use scalable indexing and distributed storage for very large résumé/job collections.
- Add real-time recruiter dashboards, notifications and candidate/job analytics.
- Evaluate algorithm performance using real datasets and report time, memory and matching-quality metrics.
- Extend team formation with additional constraints such as availability, role capacity and project priorities.

9. REFERENCES

1. LinkedIn Jobs – https://www.linkedin.com/jobs/

2. LinkedIn Help – Skills Match insight on jobs – https://www.linkedin.com/help/linkedin/answer/a793433/skills-match-insight-on-jobs

3. Indeed – https://www.indeed.com/

4. Indeed Help – Job search preferences and matching – https://www.indeed.com/help/job-seekers/articles/32603395892109-adding-or-editing-job-search-preferences

5. Workable – Recruiting and HR platform – https://www.workable.com/

6. K. H. Rosen, Discrete Mathematics and Its Applications, for foundational algorithmic concepts.

7. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.

8. The supplied project presentation: "Résumé–Job Matching & Talent Marketplace Engine", Data Structures & Algorithms – 3, Team 26, Section 12, A.Y. 2026–2027.

9. K L (Deemed to be) University, Data Structures and Algorithms – 3 Project Report template supplied for the course.